

MPOO 2027-1 · GRUPO 6 · CLASE 5 · LUNES 14 DE SEPTIEMBRE

Variables de referencia y arreglos

Hasta hoy una variable guardaba un valor. A partir de hoy hay otras que guardan una **dirección**. Esa diferencia explica los errores que más van a ver este semestre — y **es la P4 completa**, la del miércoles 23.

1 Repaso: primitivas

2 Variables de referencia

3 Arreglos

4 Cadenas

5 Wrappers

✓ **Ya entregado**

P00, P01, P02 y P03. El laboratorio de predicciones cerró el viernes 11.

➔ **Hoy vence la T01**

Radiografías con contexto, hoy a las 23:59, en entregas/apellido_nombre/tareas/t01/. Un push, sin pull request.

3 Miércoles 16

Asueto. No hay laboratorio. El siguiente es el **23 de septiembre**, con la **P4**.

Miércoles 23 · P4 Variables y arreglos. Esa práctica se cubre **hoy, completa**: variables de referencia, **arreglos**, cadenas, wrappers y argumentos por línea de comandos.

Y la que sigue: el **30 de septiembre**, la **P5** de estructuras de selección. Esa se prepara el **lunes 21**, completa: **if**, **switch**, **enum** y **Math**.

Los ocho tipos primitivos, en una lámina

TIPO	GUARDA	POR DEFECTO	TIPO	GUARDA	POR DEFECTO
byte	entero, 8 bits	0	float	decimal, 32 bits	0.0f
short	entero, 16 bits	0	double	decimal, 64 bits	0.0
int	entero, 32 bits	0	char	un carácter	' '
long	entero, 64 bits	0L	boolean	true o false	false

REGLA

Los valores por defecto son solo para atributos

Un **atributo** sin inicializar arranca con el valor de la tabla. Una **variable local** sin inicializar **no compila**: el compilador exige que le des un valor antes de usarla.

Conversión de tipos y la trampa de siempre

ENSANCHA SOLO · AUTOMÁTICO

```
int pesos = 150;
double conIva = pesos; // 150.0 · cabe, Java lo hace solo
```

ANGOSTA · LO PIDES TÚ Y PIERDES

```
double precio = 99.87;
int entero = (int) precio; // 99 · trunca, no redondea
```

La trampa número uno: división entera.

```
int a = 7, b = 2;
System.out.println(a / b); // 3 ¿y el .5?
System.out.println(a / (double) b); // 3.5
```

Si los dos operandos son enteros, el resultado es entero. Java no te avisa: te da 3 y sigue. Basta con que **uno** de los dos sea double.

AL CHAT ¿Cuánto imprime System.out.println(5 / 2 * 2.0);?

4.0 — primero 5/2 da 2 V para revelar

Dónde vive cada variable

DE INSTANCIA

Declarada en la clase, **fuera** de todo método. Una copia **por cada objeto**. Vive mientras viva el objeto.

DE CLASE

Lleva **static**. **Una sola** para todos los objetos. Existe aunque no crees ninguno.

LOCAL

Declarada dentro de un método o de un bloque { }. **Nace y muere** con el bloque.

```
public class Cuenta {
    static int total = 0;      // de clase
    double saldo;             // de instancia
    final double IVA = 0.16;  // constante

    void abonar(double monto) {
        double nuevo = saldo + monto; // local
        saldo = nuevo;
    }
}
```

REGLA

final = constante

Se le asigna un valor **una sola vez** y ya no se puede cambiar. Por convención se escribe en MAYÚSCULAS. La P4 del MADO la pide.

Si intentas reasignarla, no compila.

DEFINICIÓN

En Java hay dos tipos de variables

1 · De tipo primitivo. La variable **guarda el valor**.

2 · De referencia. La variable **guarda la dirección** de un objeto que vive en otro lado.

Es la clasificación que usa el manual de prácticas en la guía 04. No es un detalle: cambia qué pasa cuando asignas, cuando comparas y cuando pasas una variable a un método.

```
// primitiva: el valor está aquí  
int edad = 20;  
double altura = 1.75;
```

```
// de referencia: el objeto está en otro lado  
String nombre = "Ana";  
Coche mio = new Coche();  
int[] notas = new int[5];
```

Los arreglos y todos los objetos son referencias. Solo esos ocho tipos de la lámina anterior son primitivos.

Una primitiva guarda el valor, ahí mismo

```
int a = 7;
int b = a;    // se copia el VALOR
b = 99;       // ¿cuánto vale a?
```

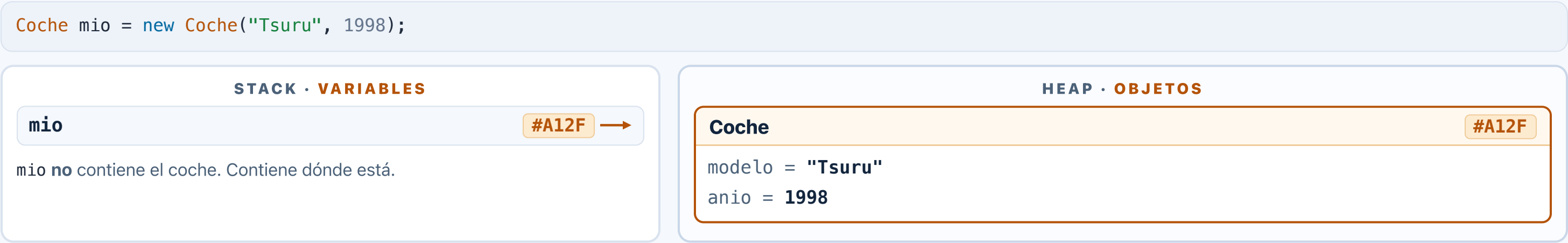
STACK · VARIABLES	
a	7
b	99

HEAP · OBJETOS

Vacío. Aquí no hay objetos:
ningún *new* se ejecutó.

a sigue valiendo 7. Son dos cajas independientes: al copiar a en b se copió el número, no la caja. Cambiar una no toca a la otra.

Una referencia guarda una dirección



El **new** hace dos cosas: construye el objeto en el **heap** y devuelve su dirección. El = guarda esa dirección en la variable.

#A12F es inventado. La dirección real nunca la ves ni te sirve — lo que importa es que **es un dato, y se puede copiar.**

DEFINICIÓN

Variable de referencia

Variable que no contiene un objeto, sino la **dirección** del objeto. Se declara con el nombre de una clase como tipo, y solo apunta a algo después de un **new** (o de recibir otra referencia).

La variable es la etiqueta; el objeto es la caja. La etiqueta se puede copiar sin copiar la caja.

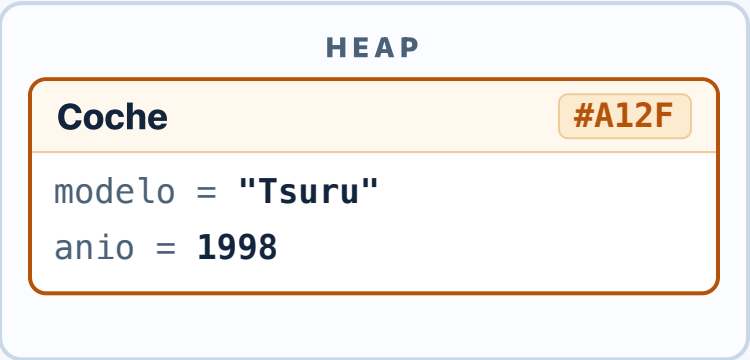
Todo lo raro que verán hoy sale de aquí: cuando copias una referencia, copias la etiqueta — **no** la caja.

El punto: seguir la flecha

```
Coche mio = new Coche("Tsuru", 1998);

mio.modelo          // leer un atributo
mio.anio = 2020;     // escribir un atributo
mio.arrancar();      // llamar un método
```

La variable mio no tiene anio. El objeto sí. El punto es el paso de una a otro.



REGLA

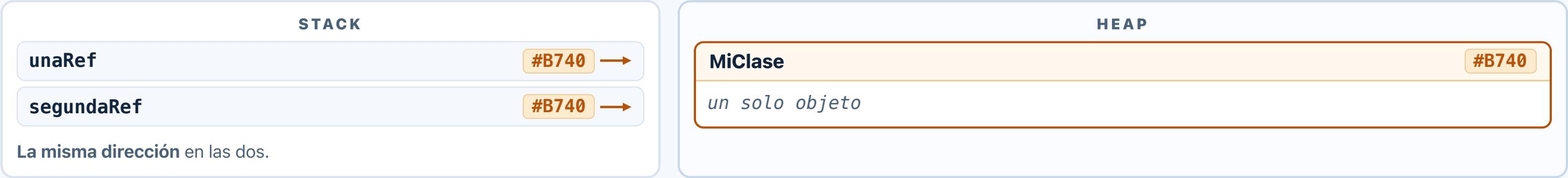
El punto no es decoración: es una instrucción

mio.anio se lee "ve a la dirección que guarda mio, y ahí adentro busca anio". Por eso solo se le puede poner punto a una referencia: un int no tiene a dónde mandarte.

Y por eso, si la referencia vale null, el punto no tiene a dónde ir. Eso es lo que sigue.

Asignar copia la dirección, no el objeto

```
MiClase unaRef = new MiClase();
MiClase segundaRef = unaRef;    // ¿cuántos objetos hay ahora?
```



Un objeto, dos nombres. Este ejemplo viene tal cual en la guía 04 del manual. La respuesta a "¿cuántos objetos hay?" es **uno** — la mayoría contesta dos.

Si toco por un nombre, se ve por el otro

```
Coche mio = new Coche("Tsuru", 1998);  
Coche prestado = mio;  
  
prestado.anio = 2020;  
  
System.out.println(mio.anio);
```

```
$ java Taller  
2020
```

Nunca tocamos mio. Pero mio y prestado son **la misma etiqueta pegada a la misma caja**.

REGLA

Copiar una referencia no copia el objeto

Para tener **dos objetos distintos** necesitas **dos new**. No hay otra forma.

AL CHAT ¿Y si en vez de `prestado.anio = 2020` escribo `prestado = new Coche(...)`?

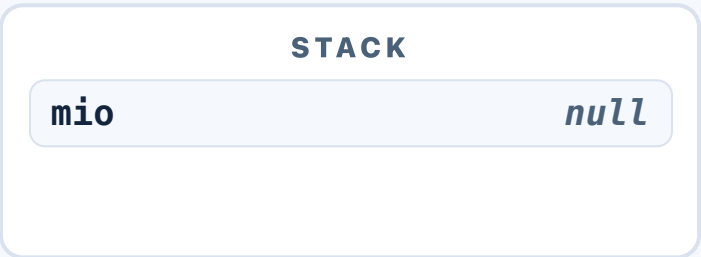
Ahí sí: mio no cambia, prestado apunta a otra caja

V para revelar

Una etiqueta pegada a nada: `null`

```
Coche mio;           // atributo: vale null
System.out.println(mio.modelo);
```

Exception in thread "main" java.lang.NullPointerException: Cannot read field "modelo" because "mio" is null



`null` es el valor por defecto de **toda** variable de referencia que sea atributo. Significa: **no apunto a ningún objeto**.

El error más común del semestre, y siempre es lo mismo: declaraste la variable pero nunca ejecutaste el `new`, o el método del que la recibiste te devolvió `null`.

DEFINICIÓN

== compara direcciones · .equals() compara contenido

Con **primitivas**, == compara valores y es lo correcto.

Con **referencias**, == pregunta "**¿son la misma caja?**", no "**¿dicen lo mismo?**".

Regla del curso: si el tipo empieza con mayúscula, se compara con .equals().

Junto con `null`, esto explica la mitad de los errores que van a traer al laboratorio.

Dos objetos idénticos no son el mismo objeto

```
Coche a = new Coche("Tsuru", 1998);
Coche b = new Coche("Tsuru", 1998);

System.out.println(a == b);      // false
System.out.println(a.equals(b)); // depende
```

STACK

a	#C001 →
b	#C002 →

HEAP

Coche	#C001
"Tsuru", 1998	
Coche	#C002
"Tsuru", 1998	

"Depende" no es una respuesta cómoda, pero es la verdad: `.equals()` heredado compara direcciones igual que `==`. Solo compara contenido si **alguien lo escribió** en la clase. `String` ya lo trae escrito; su `Coche`, no — hasta que lo escriban. Eso se ve en la unidad 2.

Java siempre pasa por valor. Siempre.

CON UNA PRIMITIVA

```
static void subir(int n) { n = n + 10; }  
  
int edad = 20;  
subir(edad);  
System.out.println(edad); // 20 · no cambió
```

Se copió el **número**. El método movió su copia.

CON UNA REFERENCIA

```
static void subir(Coche c) { c.anio = 2020; }  
  
Coche mio = new Coche("Tsuru", 1998);  
subir(mio);  
System.out.println(mio.anio); // 2020 · sí cambió
```

Se copió la **dirección**. Las dos apuntan a la misma caja.

REGLA

Se copia lo que la variable guarda

Con una primitiva se copia el valor; con una referencia se copia la dirección. En los dos casos el método recibe **una copia** — por eso puede **modificar el objeto**, pero no puede **reapuntar tu variable**.

Si dentro del método escribes `c = new Coche(...)`, afuera no pasa nada.

DEFINICIÓN

Arreglo

Colección de **tamaño fijo** de elementos **del mismo tipo**, guardados uno tras otro y accesibles por su **posición**. En Java un arreglo **es un objeto**: la variable guarda su dirección, igual que todo lo del bloque anterior.

El tamaño se decide al crearlo y ya no cambia. Si necesitas uno más grande, creas otro.

```
int[] notas = new int[5];
```

Un `new`, como con cualquier objeto. Por eso lo vemos hoy y no antes: sin referencias, un arreglo no se entiende.

Tres formas de crear uno

```
int[] a = new int[5];           // 5 posiciones, todas en 0
int[] b = {8, 9, 10, 7, 6};     // literal: tamaño y valores de un golpe
String[] c = new String[3];     // 3 posiciones, todas en null
```

SI EL TIPO ES...	ARRANCA EN
int, long, short, byte	0
double, float	0.0
boolean	false
cualquier objeto	null

REGLA

Un arreglo nuevo **nunca** está vacío

Siempre trae sus posiciones llenas con el valor por defecto del tipo. Aquí sí aplican los valores de la primera lámina, aunque sea una variable local.

El índice arranca en cero

```
int[] notas = {8, 9, 10, 7, 6};

notas[0]      // 8   · el primero
notas[4]      // 6   · el último
notas.length  // 5   · cuántos hay
notas[5]      // x   revienta
```

Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException Exception:
Index 5 out of bounds for length 5

REGLA

El último índice es `length - 1`

Cinco elementos van del 0 al 4. El 5 no existe.

length no lleva paréntesis: es un **atributo** del objeto arreglo. En cambio `"texto".length()` sí los lleva, porque ahí es un **método** de `String`. Se confunden siempre.

El molde para pasar por todos

CON ÍNDICE · CUANDO NECESITAS SABER EN CUÁL VAS

```
for (int i = 0; i < notas.length; i++) {  
    System.out.println(i + ": " + notas[i]);  
}
```

SIN ÍNDICE · CUANDO SOLO QUIERES LOS VALORES

```
for (int n : notas) {  
    System.out.println(n);  
}
```

REGLA

Hoy es un molde; el lunes 28 es un tema

El `for` completo —junto con `while` y `do-while`— se ve el **28 de septiembre**. Hoy solo necesitan copiar este molde para la **P4**: `i` arranca en `0`, sigue mientras sea `< length`, y sube de uno en uno.

`i < length`, nunca `i <= length`. Ese = de más es el desbordamiento de la lámina anterior.

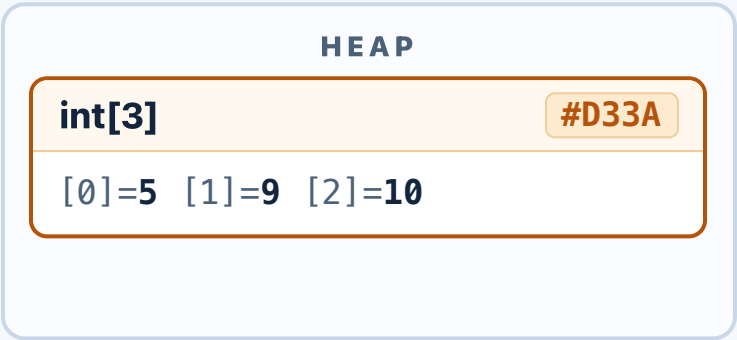
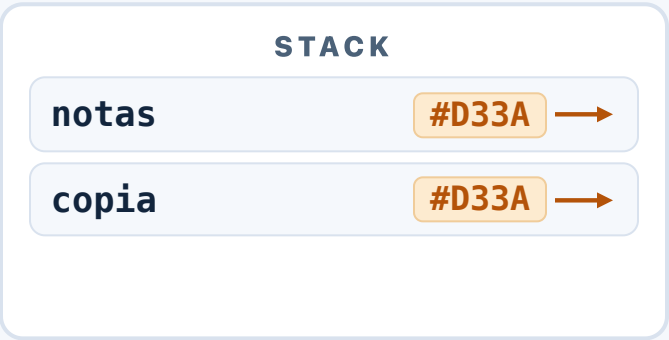
Un arreglo es una referencia, con todo lo que eso trae

```
int[] notas = {8, 9, 10};
int[] copia = notas;    // ¿copia?

copia[0] = 5;

System.out.println(notas[0]);  // 5
```

Para copiar de verdad: `int[] copia = notas.clone();` o recorrer y copiar posición por posición.



Es exactamente el aliasing de hace veinte minutos. No es una regla nueva de los arreglos: **es la misma regla de siempre**, porque el arreglo es un objeto.

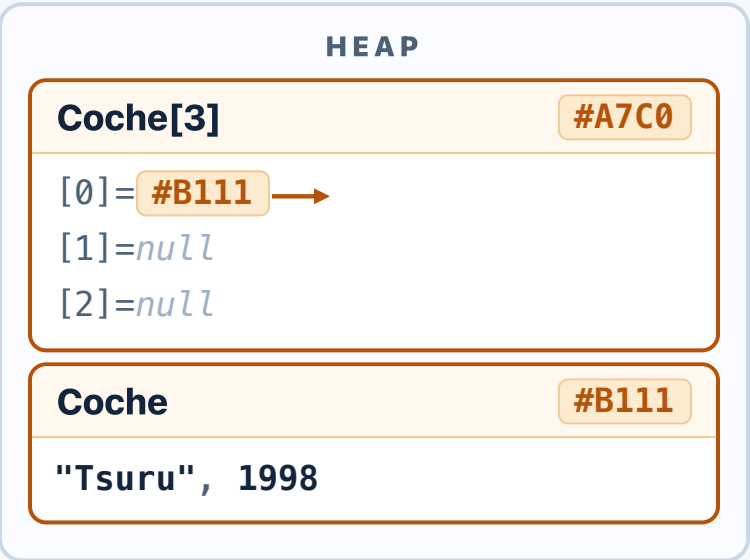
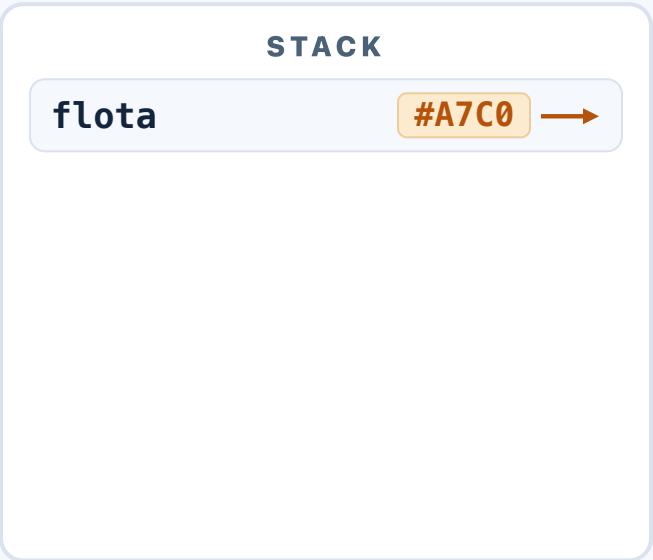
Un arreglo de objetos necesita **dos** rondas de `new`

```
Coche[] flota = new Coche[3];

flota[0].modelo;    // x NullPointerException

flota[0] = new Coche("Tsuru", 1998);
flota[0].modelo;    // ahora sí
```

El primer `new` crea **el arreglo**: tres cajones. Cada cajón arranca en `null`. El segundo `new` crea **el coche** que va en el cajón.



Viene en la guía 04 del manual, así: `MiClase[] lista = new MiClase[5];`. Eso crea **cinco null**, no cinco objetos. Faltan cinco `new` más.

Un arreglo de arreglos

```
int[][] tabla = new int[3][4];

tabla[1][2] = 7;

tabla.length      // 3 · renglones
tabla[0].length    // 4 · columnas
```

REGLA

No hay tablas en Java, hay arreglos de arreglos

Por eso el número de renglones es `tabla.length` y el de columnas es `tabla[0].length`: cada renglón **es** un arreglo con su propio `length`.

```
for (int f = 0; f < tabla.length; f++)
    for (int c = 0; c < tabla[f].length; c++)
        System.out.print(tabla[f][c]);
```

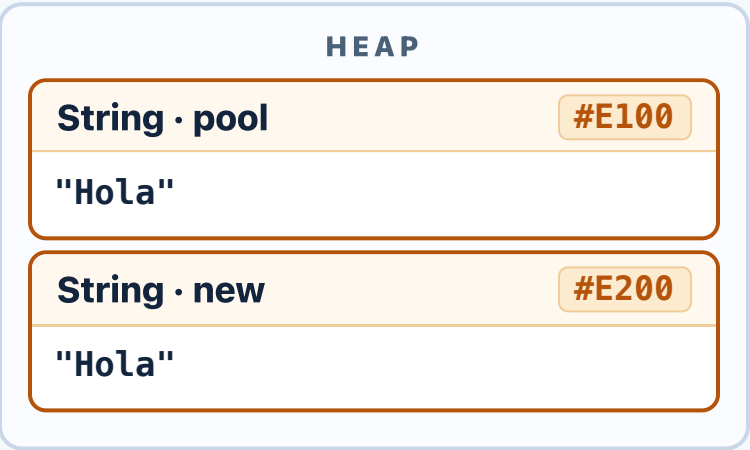
AL CHAT ¿Cuántos enteros guarda `new int[3][4]` en total?

12, todos en 0 [V para revelar](#)

Dos formas de crear un String, y no son iguales

```
String a = "Hola";
String b = "Hola";
String c = new String("Hola");

a == b      // true
a == c      // false
a.equals(c)  // true
```



Por eso == con cadenas es tan traicionero: con literales parece funcionar, porque Java los reutiliza de una zona común llamada pool. En cuanto la cadena viene de un new, del teclado o de concatenar, deja de funcionar. Siempre .equals().

DEFINICIÓN

Un **String** es inmutable

Una vez creado, su contenido **no se puede cambiar**. Todos sus métodos —`toUpperCase()`, `trim()`, `replace()`, `substring()`— **devuelven una cadena nueva** y dejan la original intacta.

Si no guardas lo que devuelven, no pasó nada.

```
String s = "hola";  
s.toUpperCase();  
System.out.println(s); // hola
```

No hizo nada. Devolvió "HOLA" y lo tiró a la basura.

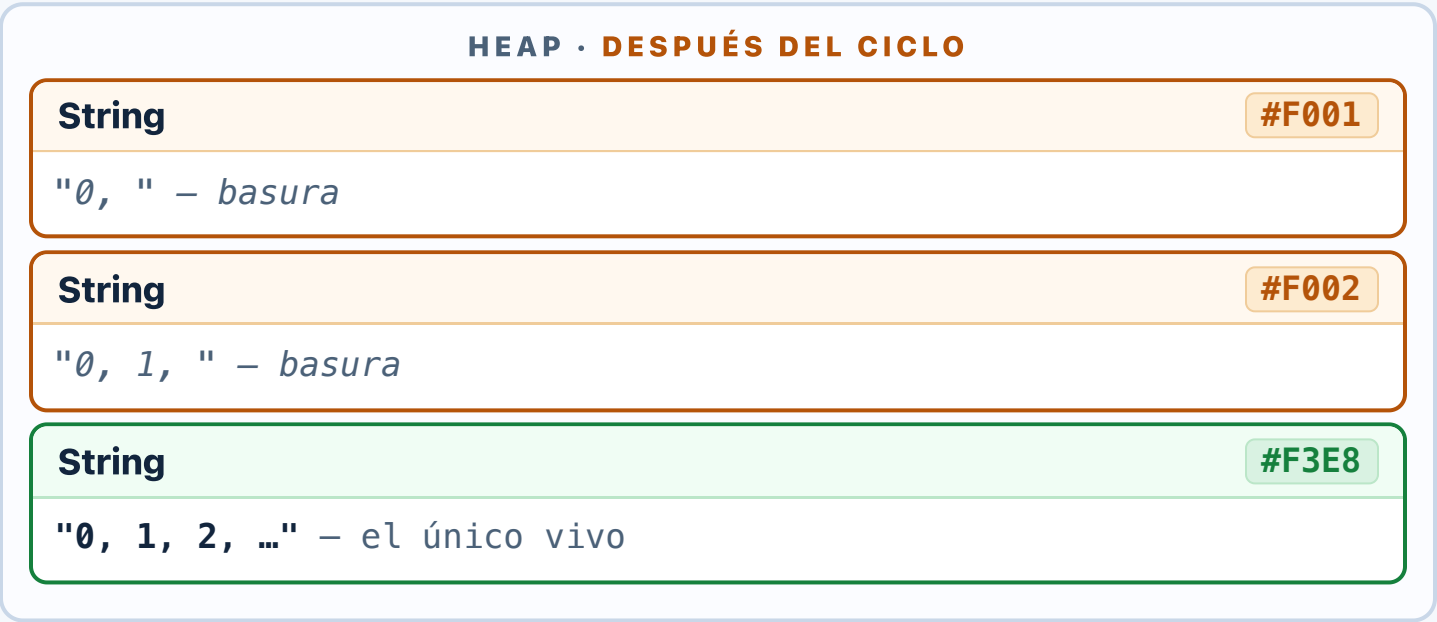
```
String s = "hola";  
s = s.toUpperCase();  
System.out.println(s); // HOLA
```

Así sí. La variable ahora apunta a la cadena nueva.

Concatenar no modifica: crea

```
String s = "";
for (int i = 0; i < 1000; i++) {
    s = s + i + ", ";
}
```

Se ve inocente. Creó **mil objetos** String, cada uno copiando entero al anterior, y tiró 999 a la basura.



Con 1000 no lo notan. Con 100 000 el programa se arrastra. **Hay una clase hecha justo para esto.**

StringBuilder: la cadena que sí se modifica

```
StringBuilder sb = new StringBuilder();

for (int i = 0; i < 1000; i++) {
    sb.append(i).append(", ");
}

String s = sb.toString(); // al final, una vez
```

	STRING	STRINGBUILDER
¿Se modifica?	No, immutable	Sí
Concatenar	crea un objeto	modifica el mismo
Para	texto que no cambia	armar texto en un ciclo

REGLA

Cuándo usar cuál

Concatenar **dos o tres** cadenas sueltas: **String** con +, se lee mejor.
Armar texto **dentro de un ciclo**: **StringBuilder**.

Los métodos de `String` que aparecen en la P4

MÉTODO	DEVUELVE
<code>length()</code>	cuántos caracteres
<code>charAt(i)</code>	el carácter en la posición <code>i</code>
<code>substring(a, b)</code>	un pedazo, de <code>a</code> a <code>b-1</code>
<code>indexOf("x")</code>	dónde aparece, o <code>-1</code>

MÉTODO	DEVUELVE
<code>equals(s)</code>	compara contenido
<code>equalsIgnoreCase(s)</code>	igual, sin distinguir mayúsculas
<code>trim()</code>	sin espacios en los extremos
<code>split(",")</code>	un <code>String[]</code> con los pedazos

Todos devuelven algo y ninguno modifica la cadena original. Si el método devuelve un valor y no lo guardas, lo perdiste. Es la misma regla de la lámina de inmutabilidad.

AL CHAT ¿Qué imprime " Ana ".trim().length()?

DEFINICIÓN

Wrapper · clase envolvente

Clase que **envuelve** un tipo primitivo para que pueda usarse donde se exige un objeto. Cada primitivo tiene la suya: `int` → `Integer`, `double` → `Double`...

Un `int` es un valor. Un `Integer` es un objeto que contiene un valor — y por lo tanto, una referencia.

¿Para qué existen? Porque hay lugares en Java donde **un primitivo no cabe**: las colecciones de la guía 06, por ejemplo, solo guardan objetos. Y porque traen herramientas útiles.

Uno por cada primitivo

PRIMITIVO	WRAPPER
byte	Byte
short	Short
int	Integer
long	Long

PRIMITIVO	WRAPPER
float	Float
double	Double
char	Character
boolean	Boolean

Dos que sí van a usar

```
int n = Integer.parseInt("42");
double d = Double.parseDouble("3.14");
```

Convierten **texto a número**. Los van a necesitar en cuanto lean algo del teclado o de la línea de comandos.

Ojo con la mayúscula

`int` con minúscula es el primitivo. `Integer` con mayúscula es la clase. **Se escriben distinto y se comparan distinto.**

Autoboxing, unboxing y la trampa que dejan

```
Integer n = 42;    // autoboxing:  int → Integer
int m = n;         // unboxing:   Integer → int
```

Java lo hace solo. El manual lo escribe así:

```
Integer n = new Integer(p); // equivale a...
Integer n = p;              // ...esto
```

Y aquí está la trampa:

```
Integer a = 127, b = 127;
System.out.println(a == b);    // true

Integer c = 128, d = 128;
System.out.println(c == d);    // false
```

REGLA

No es un capricho: son referencias

Java guarda en caché los **Integer** de -128 a 127 y los reutiliza — como el **pool** de las cadenas. Fuera de ese rango crea objetos nuevos, y `==` compara direcciones distintas.

La regla no cambia: con objetos, `.equals()`. Siempre.

Ese `String[] args` que llevan copiando desde agosto

```
public class Saludo {  
    public static void main(String[] args) {  
        System.out.println("Hola " + args[0]);  
        System.out.println("Son " + args.length);  
    }  
}
```

Si no pasas nada, `args` tiene longitud `0` y `args[0]` revienta con `ArrayIndexOutOfBoundsException`. Se revisa antes con un `if`, que es justo el tema del **lunes 21**.

```
$ java Saludo Ana Luis  
Hola Ana  
Son 2
```

`args` es **un arreglo de cadenas** — o sea, una referencia, como todo lo de hoy. Trae lo que escribiste después del nombre de la clase.

Y llegan como texto. `java Suma 3 4` te da "3" y "4", no números. Para sumarlos: `Integer.parseInt(args[0])`.

Los seis de hoy

SÍNTOMA	QUÉ PASÓ	SE ARREGLA
NullPointerException	La referencia no apunta a ningún objeto	Falta el <code>new</code> , o revisar <code>!= null</code> antes
Comparar texto nunca da true	Comparaste objetos con <code>==</code>	<code>.equals()</code>
Cambié uno y se cambiaron los dos	Copiaste la referencia, no el objeto	Un <code>new</code> por cada objeto
<code>s.toUpperCase()</code> no hace nada	<code>String</code> es inmutable	Guardar lo que devuelve: <code>s = s.toUpperCase()</code>
ArrayIndexOutOfBoundsException	Te pasaste del último índice	<code>i < length</code> , nunca <code><=</code>
NullPointerException en un arreglo de objetos	Creaste el arreglo pero no los objetos	Un <code>new</code> por cada cajón

Cuatro de los seis son el mismo error con distinta ropa: confundir la etiqueta con la caja.

Lo que se llevan

- Una primitiva guarda **el valor**; una referencia guarda **la dirección**.
- Asignar una referencia **no copia el objeto**. Dos objetos exigen dos `new`.
- Con objetos se compara con `.equals()`, nunca con `==`.
- `String` es **inmutable**: sus métodos devuelven una cadena nueva.
- Un **arreglo es un objeto**: se crea con `new`, se recorre de `0` a `length-1`, y copiarlo no lo duplica.
- Un arreglo **de objetos** necesita un `new` para el arreglo y **otro por cada elemento**.

→ Hoy, 23:59 · Tarea 1

Radiografías con contexto, en entregas/apellido_nombre/tareas/t01/. Un push a su rama, sin pull request.

21 Lunes 21 · teoría

Toda la selección: `if`, `else-if`, `switch`, `enum` y la clase `Math`. Es justo lo que pide la P5.

! Miércoles 7 de octubre · Parcial 1

Presencial, aquí en el salón. Entra **toda la unidad 1 menos repetición**: tipos, referencias, cadenas, wrappers, arreglos, ámbito y selección. Los `while` van en el segundo parcial.

Miércoles 23 · P4 Variables y arreglos, en el laboratorio — Lleguen con el JDK respondiendo y su rama al día. **Miércoles 30 · P5**, estructuras de selección.

La pregunta que resuelve casi todo lo de hoy: **¿esta variable guarda el dato, o guarda dónde está el dato?**